

`XML DATA`

JSON & XML

JSON:

- Javascript Object Notation
- It's basically just Python dictionaries that get printed out. Convenient!
- Use the `json` library to read it.

XML:

- Extensible Markup Language
- Sort of complicated tree structure of elements
- Use the `BeautifulSoup` library to read it

YAGS

(Yet Another Game Show)

Two teams of two, please....

EXAMPLE ROUND

Given the following (possibly nested) collection stored in the variable `x`, give me the Python code in terms of `x` that would be used to access a given value.

Example:

How would I access 13?

```
x = [3, 10, 15, 13, 2, 5, 1]
```

Answer:

```
x[3]
```

ROUND 1

Given the following (possibly nested) collection stored in the variable `x`, give me the Python code in terms of `x` that would be used to access a given value.

ROUND 1

Given the following (possibly nested) collection stored in the variable `x`, give me the Python code in terms of `x` that would be used to access a given value.

How would I access Josh's run time?

```
x = {"runner" : "Josh", "time": "27:15"}
```

ROUND 1

Given the following (possibly nested) collection stored in the variable `x`, give me the Python code in terms of `x` that would be used to access a given value.

How would I access Josh's run time?

```
x = {"runner" : "Josh", "time": "27:15"}
```

Answer:

```
x["time"]
```

ROUND 2

Given the following (possibly nested) collection stored in the variable `x`, give me the Python code in terms of `x` that would be used to access a given value.

ROUND 2

Given the following (possibly nested) collection stored in the variable `x`, give me the Python code in terms of `x` that would be used to access a given value.

How would I access Josh's run time?

```
x = [  
    {"runner" : "Terina", "time": "27:15"},  
    {"runner" : "Josh", "time": "24:15"},  
    {"runner" : "Rachel", "time": "25:55"},  
    {"runner" : "Jax", "time": "27:12"}  
]
```

ROUND 2

Given the following (possibly nested) collection stored in the variable `x`, give me the Python code in terms of `x` that would be used to access a given value.

How would I access Josh's run time?

```
x = [  
    {"runner" : "Terina", "time": "27:15"},  
    {"runner" : "Josh", "time": "24:15"},  
    {"runner" : "Rachel", "time": "25:55"},  
    {"runner" : "Jax", "time": "27:12"}  
]
```

```
x[1]["time"]
```

ROUND 3

Given the following (possibly nested) collection stored in the variable `x`, give me the Python code in terms of `x` that would be used to access a given value.

ROUND 3

Given the following (possibly nested) collection stored in the variable `x`, give me the Python code in terms of `x` that would be used to access a given value.

How would I access the name of the runner with a time of 25:55?

```
x = {"runners" : [  
    {"runner" : "Terina", "time": "27:15"},  
    {"runner" : "Josh", "time": "24:15"},  
    {"runner" : "Rachel", "time": "25:55"},  
    {"runner" : "Jax", "time": "27:12"}  
],  
    "race" : "CIS 1100 Classic",  
    "prize" : 1100  
}
```

ROUND 3

How would I access the name of the runner with a time of 25:55?

```
x = {"runners" : [  
    {"runner" : "Terina", "time": "27:15"},  
    {"runner" : "Josh", "time": "24:15"},  
    {"runner" : "Rachel", "time": "25:55"},  
    {"runner" : "Jax", "time": "27:12"}  
],  
    "race" : "CIS 1100 Classic",  
    "prize" : 1100  
}
```

```
x["runners"][2]["name"]
```

CHALLENGE ROUND

Get ready to write some code.

CHALLENGE ROUND

Write some code that would populate `l` with the names of all runners in the race.

```
x = {"runners" : [  
    {"runner" : "Terina", "time": "27:15"},  
    {"runner" : "Josh", "time": "24:15"},  
    {"runner" : "Rachel", "time": "25:55"},  
    {"runner" : "Jax", "time": "27:12"}  
],  
    "race" : "CIS 1100 Classic",  
    "prize" : 1100  
}  
l = []  
# TODO...
```

SOME XML TERMINOLOGY

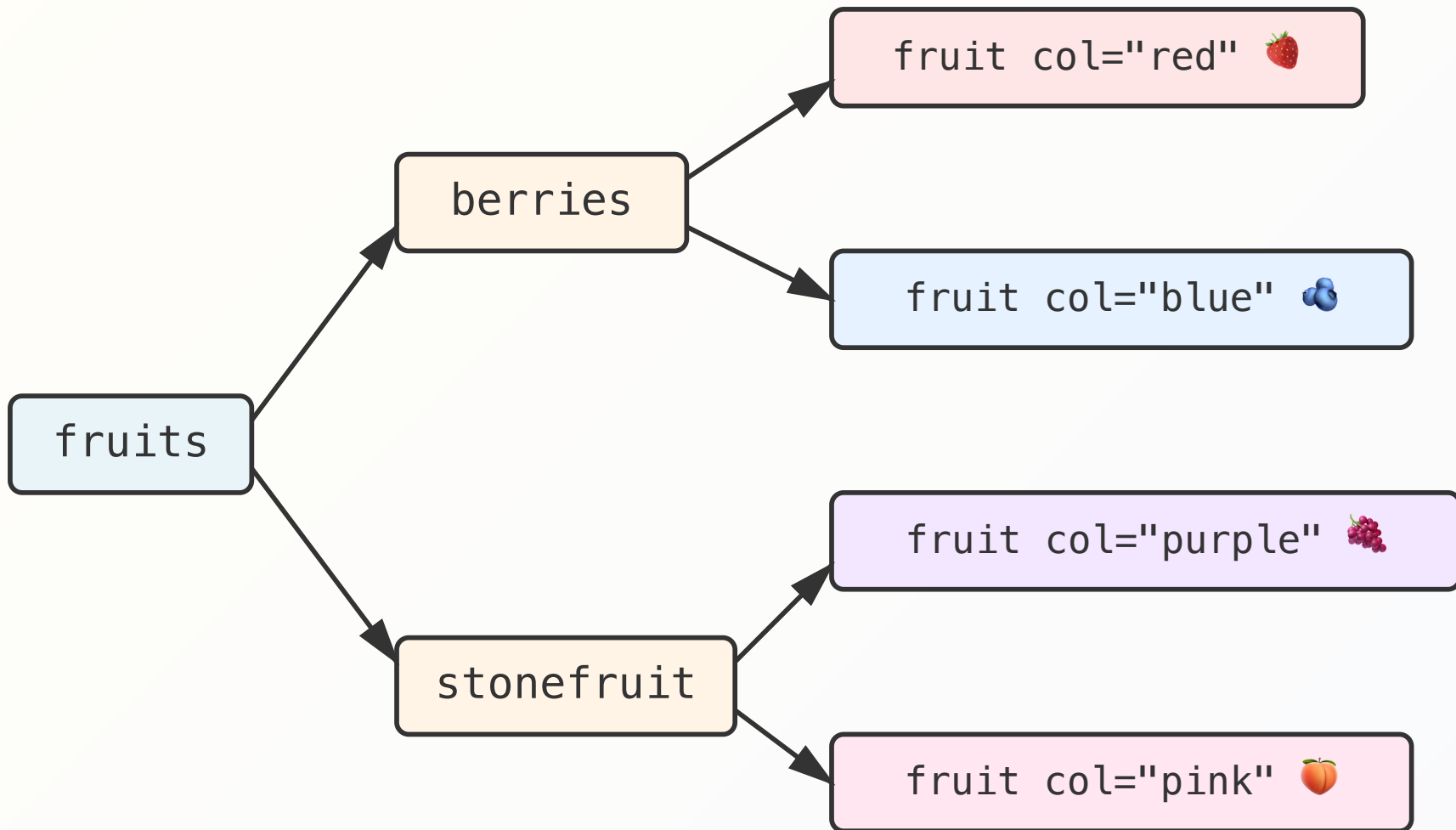
- **Elements** are the entities being represented in the XML tree, e.g. a list of fruits
- **Tags** are the names that we give to the elements, e.g. `<berries>` or `<fruit>`
- **Attributes** are properties that individual elements can have, stored in the tags, e.g. `col="blue"`
- **Text** is the information stored in the element between the tags, e.g. , 

```
<fruits>
  <berries>
    <fruit col="red">
```



```
<fruits>
  <berries>
    <fruit col="red">🍓</fruit>
    <fruit col="blue">🍇</fruit>
  </berries>
  <stonefruit>
    <fruit col="purple">🍇</fruit>
    <fruit col="pink">🍑</fruit>
  </stonefruit>
</fruits>
```

Aloud, Together How many elements? What are the different tags? How many elements have attributes?



SOME TREE TERMINOLOGY

- The **tree** is the collection of elements being represented and the connections between them
- The **root** is the element of the tree that has no ancestors.
- An **ancestor** is an element that contains another element.
 - A **parent** is a direct ancestor.
- A **descendant** is an element that is contained by another element.
 - A **child** is a direct descendant.

```
<fruits>
  <berries>
    <fruit col="red">🍓</fruit>
    <fruit col="blue">🍇</fruit>
  </berries>
  <stonefruit>
    <fruit col="purple">🍇</fruit>
    <fruit col="pink">🍑</fruit>
  </stonefruit>
</fruits>
```

Aloud, Together Which element is the root? Which elements have no children?

PARSING & TRAVERSING XML

Reading from a file:

```
from bs4 import BeautifulSoup

file = open("your_file.xml", "r")
soup = BeautifulSoup(file, "xml")
```

Downloading from internet (HW6):

```
from bs4 import BeautifulSoup
import requests as re

response = re.get(rss_url)
soup = BeautifulSoup(
    response.text,
    features="xml"
)
```

`soup` is a BeautifulSoup object we can navigate; it is a *tree* representing the full XML.

WHAT'S IN MY SOUP?

The `.prettyfy()` method returns a nicely formatted representation of the tree where all siblings are indented to the same level.

```
<fruits>
  <berries>
    <fruit col="red">
      🍓
    </fruit>
    <fruit col="blue">
      🍇
    </fruit>
  </berries>
  <stonefruit>
    <fruit col="purple">
```

An important skill in programming is *pattern matching*.

- Try to predict what you think a method would do in a given context based on its name.
- Recognize familiar syntax in unfamiliar applications and reason about how it works.

SEARCHING AROUND

Recall: `element.find()` and `element.find_all()` return the first/all matching tag(s) that are descendants of `element`.

```
from bs4 import BeautifulSoup

file = open("fruits.xml", "r")
soup = BeautifulSoup(file, "xml")
print(soup.find("fruit"))
```



```
<fruit col="red">🍓</fruit>
```

```
<fruits>
  <berries>
    <fruit col="red">🍓</fruit>
    <fruit col="blue">🍇</fruit>
  </berries>
  <stonefruit>
    <fruit col="purple">🍇</fruit>
    <fruit col="pink">🍑</fruit>
  </stonefruit>
</fruits>
```

SEARCHING AROUND

Recall: `element.find()` and `element.find_all()` return the first/all matching tag(s) that are descendants of `element`.

```
from bs4 import BeautifulSoup

file = open("fruits.xml", "r")
soup = BeautifulSoup(file, "xml")
all_fruits = soup.find_all("fruit")
print(len(all_fruits))
```



4

```
<fruits>
  <berries>
    <fruit col="red">🍓</fruit>
    <fruit col="blue">🍇</fruit>
  </berries>
  <stonefruit>
    <fruit col="purple">🍇</fruit>
    <fruit col="pink">🍑</fruit>
  </stonefruit>
</fruits>
```


SEARCHING AROUND

Recall: `element.find()` and `element.find_all()` return the first/all matching tag(s) that are descendants of `element`.

```
from bs4 import BeautifulSoup

file = open("fruits.xml", "r")
soup = BeautifulSoup(file, "xml")
all_fruits = soup.find_all("fruit")
print(all_fruits[3])
```



S7 What's printed?

```
<fruits>
  <berries>
    <fruit col="red">🍓</fruit>
    <fruit col="blue">🍇</fruit>
  </berries>
  <stonefruit>
    <fruit col="purple">🍇</fruit>
    <fruit col="pink">🍑</fruit>
  </stonefruit>
</fruits>
```

SEARCHING AROUND

Can also filter by attribute by passing a dictionary as a second argument to `element.find()` and `element.find_all()`.

```
from bs4 import BeautifulSoup

file = open("fruits.xml", "r")
soup = BeautifulSoup(file, "xml")
blue_fruit = soup.find_all("fruit",
print(len(blue_fruit))
```

S8 What's printed?

```
<fruits>
  <berries>
    <fruit col="red">🍓</fruit>
    <fruit col="blue">🍇</fruit>
  </berries>
  <stonefruit>
    <fruit col="purple">🍇</fruit>
    <fruit col="pink">🍑</fruit>
  </stonefruit>
</fruits>
```

ACCESSING ATTRIBUTES

If you have an element and you want to know the value associated with a certain attribute, treat it like a dictionary.

```
from bs4 import BeautifulSoup

file = open("fruits.xml", "r")
soup = BeautifulSoup(file, "xml")
a_fruit = soup.find_all("fruit")[2]
print(a_fruit["col"])
```

S9 What's printed?

```
<fruits>
  <berries>
    <fruit col="red">🍓</fruit>
    <fruit col="blue">🍇</fruit>
  </berries>
  <stonefruit>
    <fruit col="purple">🍇</fruit>
    <fruit col="pink">🍑</fruit>
  </stonefruit>
</fruits>
```

ACCESSING TEXT

If you have an element and you want the text inside of it, use `.text`

```
from bs4 import BeautifulSoup

file = open("fruits.xml", "r")
soup = BeautifulSoup(file, "xml")
a_fruit = soup.find_all("fruit")[3]
print(a_fruit.text)
```

S10 What's printed? 🎨 🖌️

```
<fruits>
  <berries>
    <fruit col="red">🍓</fruit>
    <fruit col="blue">🍇</fruit>
  </berries>
  <stonefruit>
    <fruit col="purple">🍇</fruit>
    <fruit col="pink">🍑</fruit>
  </stonefruit>
</fruits>
```

FINISH THE SNIPPET

(C12) Finish the snippet to print a list of all colors of all the fruits inside of a given file `fruits.xml` that has the same structure as before. (Should work for any XML with same structure.)

```
from bs4 import BeautifulSoup

file = open("fruits.xml", "r")
soup = BeautifulSoup(file, "xml")
... # TODO!
```

```
<fruits>
  <berries>
    <fruit col="red">🍓</fruit>
    <fruit col="blue">🍇</fruit>
  </berries>
  <stonefruit>
    <fruit col="purple">🍇</fruit>
    <fruit col="pink">🍑</fruit>
  </stonefruit>
</fruits>
```

KNOW YOUR ELEMENTS

Given an element, use `.name`, `.attrs`, and `.text` to get the tag name, attributes dictionary, and text of that element.

```
a_fruit = soup.find("fruit")  
print(f"{a_fruit.name}, {a_fruit.attrs}, {a_fruit.text}")
```

```
fruit, {'color': 'red'}, 🍓
```

SEARCH CHILDREN, NOT DESCENDANTS

To limit a search to *just children*, use `recursive=False` as a keyword argument to `find()` / `find_all()`.

```
from bs4 import BeautifulSoup

file = open("canada.xml", "r")
soup = BeautifulSoup(file, "xml")
canada = soup.find("canada")
leaders = canada.find_all("leader",
print(leaders)
```

```
[<leader title="Prime
Minister">Mark Carney</leader>]
```

```
<canada>
  <leader title="Prime Minister">
    Mark Carney
  </leader>
  <province name="Ontario">
    <leader title="Premier">
      Doug Ford
    </leader>
  </province>
  ...
</canada>
```

ANNOYING NOTE

Technically, the root of the tree is usually an invisible `document` element with the real root as its only child. So, for `recursive=False` searches, make sure you know exactly where you are.

```
from bs4 import BeautifulSoup

file = open("canada.xml", "r")
soup = BeautifulSoup(file, "xml")
leaders = soup.find_all("leader", recursive=False)
print(soup.name, leaders)
```

```
document, []
```


ANNOYING NOTE

Technically, the root of the tree is usually an invisible `document` element with the real root as its only child. So, for `recursive=False` searches, make sure you know exactly where you are.

```
from bs4 import BeautifulSoup

file = open("canada.xml", "r")
soup = BeautifulSoup(file, "xml")
canada = soup.find("canada")
leaders = canada.find_all("leader", recursive=False)
print(canada.name, leaders)
```

```
canada, [<leader title="Prime Minister">Mark Carney</leader>]
```